664245044 เมทิดา ตั้งเกียรติกร 66/44

1.กำหนด Node และ Relationship ที่จำเป็นสำหรับระบบชมรม

คำอธิบายเส้นทางความสัมพันธ์:

- ระบบเริ่มต้นจากการสร้างโหนดหลัก 2 ประเภท คือ `Student` (ตัวแทนนักเรียน) และ `Club` (ตัวแทนชมรม) ซึ่งโหนดแต่ละตัวจะมี Property เฉพาะตัว เช่น `student_id` และ `name` เพื่อระบุตัวตน
- จากนั้นสร้างเส้นความสัมพันธ์ (Relationship) เชื่อมโยงระหว่างโหนด ได้แก่:
    - `[:FRIEND_OF]` เชื่อมโยงระหว่างโหนด `Student` กับ `Student` เพื่อแสดงเครือข่ายความเป็นเพื่อน
    - `[:MEMBER_OF]` เชื่อมโยงจากโหนด `Student` ไปยังโหนด `Club` เพื่อระบุว่านักเรียนคนนั้นสังกัดหรือเป็นสมาชิกชมรมใดอยู่

Node Labels:
- Student (ตัวแทนนักเรียน, Property: student_id, name)
- Club (ตัวแทนชมรม, Property: club_id, name)

Relationships:
- [:FRIEND_OF] เชื่อมระหว่าง Student กับ Student (การเป็นเพื่อนกัน)
- [:MEMBER_OF] เชื่อมจาก Student ไปยัง Club (การเป็นสมาชิกชมรม)

Recommendation Goal: แนะนำ Club ที่เพื่อนเป็นสมาชิกหรือสอดคล้องกับเครือข่ายความสัมพันธ์ของนักเรียน

2.สร้างข้อมูลตัวอย่าง (Student 5 คน และ Club 3 ชมรม)

คำอธิบายเส้นทางความสัมพันธ์:

- เป็นการจำลองสถานการณ์จริงในโรงเรียนโดยกำหนดให้มีนักเรียน 5 คน (`S001` ถึง `S005`) และชมรม 3 ชมรม (`C01` ถึง `C03`)
- กำหนดเส้นทางความสัมพันธ์ว่า **นักเรียน S001 เป็นเพื่อน (`FRIEND_OF`) กับ S002, S003 และ S004**
- กำหนดเส้นทางสมาชิกภาพ (`MEMBER_OF`) ว่า S001 อยู่ชมรม Board Game Club (`C03`), ในขณะที่กลุ่มเพื่อน (S002, S003, S004) กระจายกันไปเป็นสมาชิกของ Coding Club (`C01`) และ Robotics Club (`C02`)

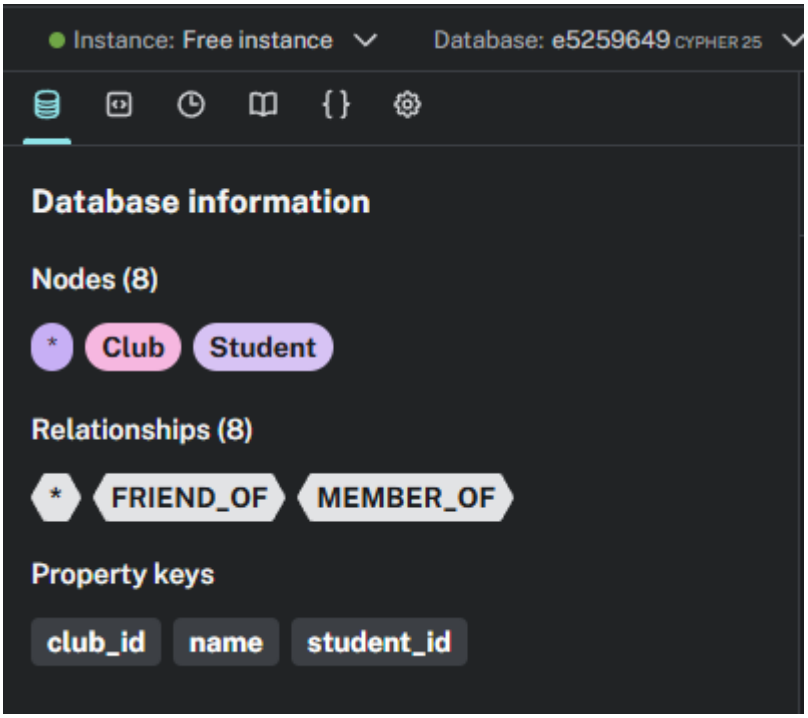
```
CREATE
// สร้าง Student 5 คน
(s1:Student {student_id: 'S001', name: 'Anan'}),
(s2:Student {student_id: 'S002', name: 'Mali'}),
(s3:Student {student_id: 'S003', name: 'Somchai'}),
(s4:Student {student_id: 'S004', name: 'Nida'}),
(s5:Student {student_id: 'S005', name: 'Kanda'}),

// สร้าง Club 3 ชมรม
```

```
(c1:Club {club_id: 'C01', name: 'Coding Club'}),
(c2:Club {club_id: 'C02', name: 'Robotics Club'}),
(c3:Club {club_id: 'C03', name: 'Board Game Club'}),

// สร้างความเป็นเพื่อน (S001 เป็นเพื่อนกับ S002, S003, S004)
(s1)-[:FRIEND_OF]->(s2),
(s1)-[:FRIEND_OF]->(s3),
(s1)-[:FRIEND_OF]->(s4),

// สร้างการเป็นสมาชิกชมรม
(s1)-[:MEMBER_OF]->(c3), // S001 อยู่ Board Game Club แล้ว
(s2)-[:MEMBER_OF]->(c1), // เพื่อน S002 อยู่ Coding Club
(s2)-[:MEMBER_OF]->(c2), // เพื่อน S002 อยู่ Robotics Club
(s3)-[:MEMBER_OF]->(c1), // เพื่อน S003 อยู่ Coding Club
(s4)-[:MEMBER_OF]->(c2); // เพื่อน S004 อยู่ Robotics Club
```



3.ชุดคำสั่ง Query สำหรับการแนะนำชมรม
3.1 ใช้ MATCH + WHERE + RETURN เพื่อดูชมรมของเพื่อน S001

คำอธิบายเส้นทางความสัมพันธ์:

- เริ่มต้นการเดินทางที่โหนดตัวเรา (me) ที่มีรหัส S001
- เดินทางผ่านเส้นทาง [:FRIEND_OF] ไปยังโหนดเพื่อน (friend)
- จากนั้นเดินทางต่อผ่านเส้นทาง [:MEMBER_OF] ไปยังโหนดชมรม (c) ที่เพื่อนคนนั้นสังกัดอยู่
- ผลลัพธ์จะแสดงรายชื่อเพื่อนของ S001 และชมรมที่เพื่อนแต่ละคนเข้าร่วม เพื่อดูว่าคนรอบตัว S001 สนใจทำกิจกรรมอะไรบ้าง

```
MATCH (me:Student)-[:FRIEND_OF]-(friend:Student)-[:MEMBER_OF]->(c:Club)
WHERE me.student_id = 'S001'
RETURN friend.name AS friend_name, c.name AS club_name;
```

```
e5259649$ MATCH (me:Student)-[:FRIEND_OF]-(friend:Student)-[:MEMBER_OF]->(c:Club) WHERE me.student_id
```

Table Raw

	friend_name	club_name
1	"Mali"	"Coding Club"
2	"Mali"	"Robotics Club"
3	"Somchai"	"Coding Club"
4	"Nida"	"Robotics Club"

3.2 ใช้ Traversalไปยัง Club และใช้ count(DISTINCT friend) ให้คะแนน

คำอธิบายเส้นทางความสัมพันธ์:

- เป็นการเดินทางข้ามโหนดแบบต่อเนื่อง (Multi-hop Traversal) เริ่มจาก S001 → ผ่านเพื่อน (friend) → ไปสิ้นสุดที่ชมรม (c)
- ระบบจะทำการนับจำนวนเพื่อนที่ไม่ซ้ำกัน (count(DISTINCT friend)) ที่อยู่ในแต่ละชมรม เพื่อนำมาคำนวณเป็น **คะแนนความนิยม (friend_score)** และเรียงลำดับจากมากไปน้อย ทำให้รู้ว่าชมรมไหนมีเพื่อนของ S001 ไปอยู่รวมกันมากที่สุด

```
MATCH (me:Student {student_id:
'S001'})-[:FRIEND_OF]-(friend:Student)-[:MEMBER_OF]->(c:Club)
RETURN c.club_id, c.name, count(DISTINCT friend) AS friend_score
ORDER BY friend_score DESC;
```

```
$ Start typing or generate with AI (ctrl + I)

e5259649$ MATCH (me:Student {student_id: 'S001'})-[:FRIEND_OF]-(friend:Student)-[:MEMBER_OF]->(c:Club) RETURN c.club_
```

Table Raw Search table...

	c.club_id	c.name	friend_score
1	"C01"	"Coding Club"	2
2	"C02"	"Robotics Club"	2

Started streaming 2 re

3.3 ตัด Club ที่ S001 เป็น MEMBER_OF อยู่แล้วออก แล้วสร้าง Recommendation (Query แบบครบสายความคิด)

คำอธิบายเส้นทางความสัมพันธ์:

- เส้นทางนี้ต่อยอดจากการหาคะแนนในข้อก่อนหน้า แต่เพิ่มเงื่อนไขกรอง (`WHERE NOT EXISTS`) เข้ามาตรวจสอบเส้นทางตรงระหว่างตัวเรา (`me`) กับชมรม (`c`) ว่ามีเส้นทาง `[:MEMBER_OF]` อยู่แล้วหรือไม่
- หากพบว่า S001 เป็นสมาชิกชมรมนั้นอยู่แล้ว (เช่น Board Game Club) ระบบจะทำการตัดออกจากรายการทันที เพื่อป้องกันไม่ให้แนะนำชมรมที่นักเรียนสังกัดอยู่แล้วซ้ำซ้อน
- ผลลัพธ์สุดท้ายจึงเป็นการแนะนำชมรมใหม่ที่ตรงกับพฤติกรรมความสนใจของเพื่อนฝูงมากที่สุด

```
MATCH (me:Student {student_id:
'S001'})-[:FRIEND_OF]-(friend:Student)-[:MEMBER_OF]->(c:Club)
WHERE NOT EXISTS {
   MATCH (me)-[:MEMBER_OF]->(c)
}
RETURN c.club_id, c.name AS recommended_club, count(DISTINCT friend) AS
friend_score
ORDER BY friend_score DESC, c.name;
```

```
e5259649$ MATCH (me:Student {student_id: 'S001'})-[:FRIEND_OF]-(friend:Student)-[:MEMBER_OF]->(c:Club) WHERE NOT EX
```

Table Raw Search table...

	c.club_id	recommended_club	friend_score
1	"C01"	"Coding Club"	2
2	"C02"	"Robotics Club"	2

Started streaming 2

4. คำอธิบายภาษาไทย (ทำไมระบบจึงแนะนำ Club อันดับ 1)
ระบบแนะนำ Coding Club ให้กับนักเรียน S001 เนื่องจากมีเพื่อนสนิทของ S001 (ได้แก่ Mali และ Somchai) เข้าร่วมเป็นสมาชิกมากที่สุดเป็นอันดับแรก โดยระบบได้ตรวจสอบและตัดชมรมเดิมที่ S001 เคยเป็นสมาชิกอยู่แล้ว (Board Game Club) ออกไป เพื่อคัดเลือกเฉพาะชมรมใหม่ที่มีเพื่อนเข้าร่วมมากที่สุดมานำเสนอ